

Question 1)

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your program. For each heuristic: 1. Give a precise formula or pseudocode. 2. Explain what unavoidable remaining cost it lower-bounds. 3. Explain why it cannot overestimate the true optimal remaining cost. Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

Steps heuristic = $h_steps(n) = d(n)$

- counts how many positions in the current word are different from the goal.
- one legal action changes only one character so every incorrect position must be changed at least once
- This means that if $d(n)$ mismatched positions, at least $d(n)$ actions are required to reach the goal.
- every action costs 1 under the steps cost measure so the true remaining cost must be at least $d(n)$.
- So: $h_steps(n)$ cannot overestimate the true optimal remaining cost so it is admissible.

Scrabble heuristic = $h_scrabble(n)$ = sum of the Scrabble values of the goal letters in every mismatched position.

- Example: dog $\rightarrow$ cat.
 - o 3 different positions therefore ‘c’ ‘a’ ‘t’ need to be inserted which have a cost of $3 + 1 + 1 = 5$
 - o This is a lower bound since every incorrect position must eventually have the correct goal letter so the cost of 5 to insert them is inevitable no matter what. If we were to make other changes instead of inserting the goal letters then the cost would only increase. So the heuristic can't over estimate the true remaining Scrabble cost therefore admissible

Frequency heuristic = $h_frequency(n) = 0$, if n is the goal. Else: $h_frequency(n) = d(n) + rarity(goal)$

- Cost of entering a word is $1 + rarity(word)$ and rarity values can't be negative
- If $d(n)$ positions aren't the goal positions, then we need at least $d(n)$ actions because each action changes only 1 char. Since every action has a min cost of 1 the remaining path must cost at least $d(n)$

- Since every solution must eventually have the goal word the rarity value of the goal word must be used on the final action no matter what. So the cost must be atleast $d(n) + \text{rarity}(\text{goal})$
- If we are at the goal there is no remaining cost so we return 0
- So frequency heuristic cannot overestimate therefore admissible

Question 2)

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554065	808	38.8554065	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.240228	1084	38.240228	302	72.14%

UCS and A* returned same solution cost for all tests.

WHY → ASK: both returned costs of 5 for steps, 9 for Scrabble, and 38.85540652 for frequency.

BEAR → DUCK: both returned costs of 4 for steps, 11 for Scrabble, and 38.2402284 for frequency.

Consistent with the heuristics being admissible since A* should still return an optimal cost solution when using an admissible heuristic.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

WHY → ASK: Scrabble heuristic: 98.30%

BEAR → DUCK: Scrabble heuristic: 99.80%

A* expanded fewer nodes than UCS when using the Scrabble heuristic in both cases (by a lot).

3. For each word pair, under which cost measure was the heuristic least informative?

Frequency heuristic was the least informative.

WHY → ASK: frequency only reduced expansions by: 21.53% (98% for steps and Scrabble).

BEAR → DUCK: frequency reduced expansions by: 72.14% (~99% for steps and Scrabble)

4. Explain the differences using the definitions of your heuristics, not runtime alone.

The steps heuristic counts how many letter positions still differ from goal. Each action can change only one position so we have a direct lower bound of the number of remaining actions.

Scrabble heuristic is more informative because it identifies which positions are incorrect and uses Scrabble values of the goal letters that must eventually be inserted. A* has more information about the unavoidable remaining cost which is why it has the largest reduction in expanded nodes.

frequency heuristic less informative because it only uses the number of mismatched positions and the rarity of the goal word. Doesn't account for rarity costs of intermediate words that could be apart of the solution path (these intermediate values can be a big part in the true remaining cost therefore we underestimate it by larger amount). A* has less info for knowing the better paths and expands more nodes.

Question 3)

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal. For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from $W1$ to $W2$ must always equal the optimal cost from $W2$ to $W1$. For each cost measure:

1. State yes or no.

2. Justify your conclusion.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary

Steps: Yes, the optimal cost from $w1$ to $w2$ is always equal to the optimal cost from $w2$ to $w1$. Since every legal action costs 1, if there is a path from $w1$ to $w2$ containing x amount of actions, then reversing that path gives a path from $w2$ to $w1$ that contains the same amount of x actions so it's the same cost.

Scrabble: no, cost of an action depends on the letter being inserted so when you reverse the action you are inserting different letters with potentially different costs. Ex: dog→hog, d changes to h which is a cost of 4 so $\text{cost}(\text{dog}, \text{hog}) = 4$ while for hog→dog, h changes to d which has a cost of 2 so $\text{cost}(\text{hog}, \text{dog}) = 2$ so they aren't equal.

Frequency: no, cost is measured by $1 + \text{rarity}(\text{word})$ so if you reverse the action the words rarity will be different. Example lets say rarity of dog is 5 while rarity of hog is 11. This means that $\text{cost}(\text{dog}, \text{hog}) = 1 + 11 = 12$ while $\text{cost}(\text{hog}, \text{dog}) = 1 + 5 = 6$ so different costs for reverse.

All of these actions were reversible but for scrabble and frequency, the reversible action did not equal symmetric action cost.

Question 4)

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

Chatgpt, GPT-5.6

A good way to improve h_0 is to account for the fact that **every future word you enter costs at least 1, plus whatever rarity is unavoidable.**

Let the current state be n , the goal be g , and

$$D(n, g) = \{i: n_i \neq g_i\}.$$

The baseline is simply

$$h_0(n) = |D(n, g)|.$$

For each mismatched position i , consider all dictionary words that have the **correct goal letter in position i** :

$$S_i = \{w \in \text{dictionary}: w_i = g_i\}.$$

Define

$$r_i = \min_{w \in S_i} \text{rarity}(w).$$

Then use

$$h(n) = h_0(n) + \max_{i \in D(n, g)} r_i$$

with $h(g) = 0$.

Why this is admissible

Any path from n to the goal must eventually correct **every mismatched position**. In particular, for each $i \in D(n, g)$, the path must enter some word whose i -th letter equals the goal's i -th letter.

Entering that word incurs

$$1 + \text{rarity}(w).$$

The ordinary $h_0(n)$ already accounts for the unavoidable base cost of at least one letter-changing action per mismatched position. In addition, the path must incur rarity of at least r_i for every particular position i . Therefore it certainly incurs at least

$$\max_{i \in D(n, g)} r_i$$

in rarity cost. Thus

Let $D(n, g)$ be the set of positions where the current state n differs from the goal g .

The baseline heuristic is:

$h_0(n)$ = number of positions where n differs from g

For every mismatched position i , define r_i as the minimum rarity of any dictionary word that has the correct goal letter in position i .

Then the proposed heuristic is:

$$h(n) = h_0(n) + \max(r_i)$$

where the maximum is taken over all mismatched positions. If n is already the goal, $h(n) = 0$.

2. A precise formula or pseudocode for the proposed heuristic.

D = all positions i where $state[i] \neq goal[i]$

if D is empty:

return 0

for each i in D :

r_i = minimum rarity of any legal word w where $w[i] == goal[i]$

return $len(D) + \max(r_i \text{ for } i \text{ in } D)$

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

Admissibility proof by contradiction:

Assume proposed heuristic overestimates the true optimal remaining cost for some state n .

Let $d = h_0(n)$ be the number of mismatched positions.

Let r_i be minimum rarity of any legal word having the correct letter at mismatched position i .

Now we can choose mismatched position i with the largest r_i .

Any path to the goal must make at least d actions because one action changes only one letter.

The path must also eventually enter a word that has the correct letter at position i , and that word has rarity at least r_i

so every solution must cost at least $d + r_i = h_0(n) + \max r_i = h(n)$

this is a contradiction since we assumed that $h(n)$ is larger than the true optimal remaining cost so the heuristic is admissible.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

Not needed

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

tested both h_0 and the proposed rarity aware heuristic using A* on the two required frequency-cost problems.

WHY → ASK

h_0 :

Solution cost = ~ 38.8554

Nodes expanded = 755

Proposed rarity-aware heuristic:

Solution cost = ~ 38.8554

Nodes expanded = 635

Reduction in nodes expanded:

$$100 * (755 - 635) / 755 = 15.89\%$$

BEAR → DUCK

h_0 :

Solution cost = ~ 38.240

Nodes expanded = 720

Proposed rarity-aware heuristic:

Solution cost = ~ 38.240

Nodes expanded = 574

Reduction in nodes expanded:

$$100 * (720 - 574) / 720 = 20.28\%$$

proposed rarity-aware heuristic returned the same optimal solution cost as h_0 for both tested frequency cost problems while expanding fewer nodes.

WHY → ASK: # of expanded nodes decreased from 755 to 635 (15.89% reduction)

BEAR → DUCK: # of expanded nodes decreased from 720 to 574 (20.28%)

These experiments support using the proposed heuristic instead of h_0 because it gave A* more information and reduced the number of expanded nodes without changing the optimal solution cost.

AI Disclosure: I used AI to refresh me on A* vs UCS and heuristics. Also